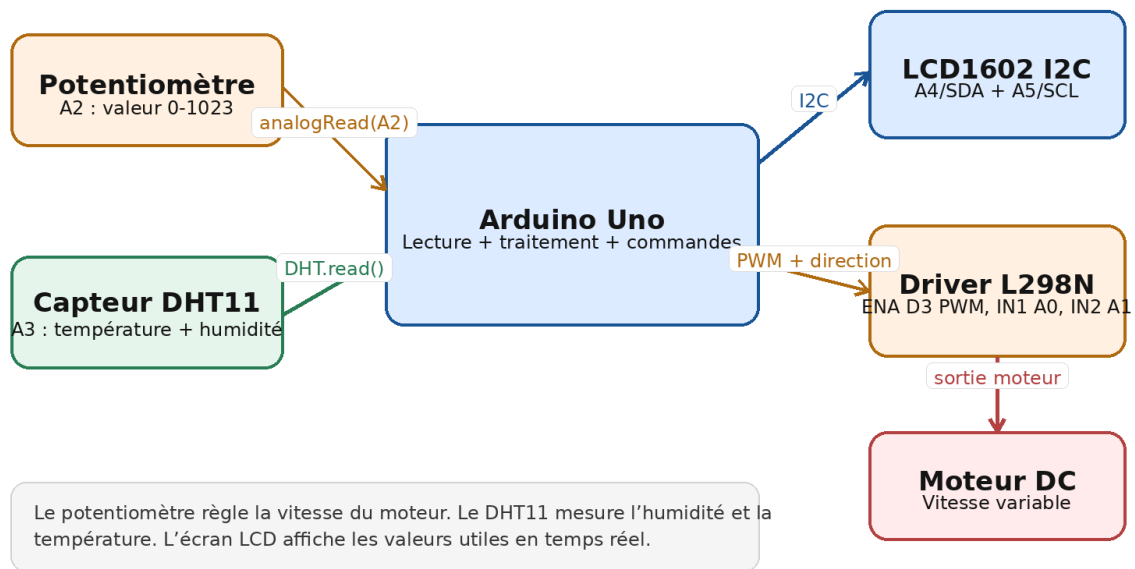


DOCUMENTATION TECHNIQUE

Contrôle d'un moteur DC avec potentiomètre, DHT11 et écran LCD1602 I2C

Arduino Uno + L298N + LCD1602 + DHT11

Schéma fonctionnel du montage



Travail réalisé en binôme

Abdelkhalek Hanbel - Hafida Belayd

Date : 21/05/2026

1. Introduction

Ce document présente la documentation technique d'un montage Arduino Uno réalisé en binôme. Le système permet de contrôler la vitesse d'un moteur DC à l'aide d'un potentiomètre, de mesurer la température et l'humidité avec un capteur DHT11, puis d'afficher les informations principales sur un écran LCD1602 avec interface I2C.

L'objectif est de montrer l'intégration de plusieurs composants électroniques dans un seul programme : lecture analogique, commande PWM, acquisition de données environnementales et affichage local en temps réel.

Type de projet	Carte utilisée	Entrées	Sorties
Système embarqué Arduino	Arduino Uno	Potentiomètre + DHT11	Moteur DC + LCD1602

2. Objectifs du montage

- Lire la valeur du potentiomètre sur l'entrée analogique A2.
- Convertir la valeur 0-1023 en une consigne PWM de 0 à 255.
- Contrôler la vitesse du moteur DC à travers le module L298N.
- Lire la température et l'humidité du capteur DHT11 toutes les 2 secondes.
- Afficher la vitesse PWM, la température et l'humidité sur l'écran LCD1602 I2C.
- Afficher les mêmes informations dans le moniteur série pour faciliter le test et le diagnostic.

3. Matériel utilisé

Composant	Rôle dans le montage
Arduino Uno	Carte microcontrôleur principale. Elle lit les entrées, exécute le programme et commande les sorties.
LCD1602 I2C	Écran 16 colonnes x 2 lignes. Il affiche la vitesse, la température et l'humidité.
Capteur DHT11	Capteur de température et d'humidité utilisé pour mesurer les conditions ambiantes.
Module L298N	Driver moteur utilisé pour piloter un moteur DC avec une commande de direction et une commande PWM.
Moteur DC	Actionneur dont la vitesse est contrôlée par le signal PWM.
Potentiomètre	Entrée analogique permettant de régler manuellement la vitesse du moteur.
Breadboard et fils Dupont	Éléments de connexion pour réaliser le montage sans soudure.

4. Branchement et affectation des broches

Le programme utilise les broches suivantes. Les broches analogiques A0 et A1 sont utilisées comme sorties numériques pour commander les entrées IN1 et IN2 du module L298N.

Module	Broche module	Broche Arduino	Fonction
LCD1602 I2C	SDA	A4	Communication I2C
LCD1602 I2C	SCL	A5	Communication I2C
DHT11	DATA	A3	Lecture température/humidité
Potentiomètre	Signal central	A2	Lecture analogique 0-1023
L298N	ENA	D3	Signal PWM pour contrôler la vitesse
L298N	IN1	A0	Commande de direction
L298N	IN2	A1	Commande de direction
Alimentation	5V / GND	5V / GND	Alimentation logique et masse commune

Remarque de sécurité : toutes les masses doivent être communes. Le moteur doit être alimenté selon ses besoins électriques et selon les limites du module L298N.

5. Schéma fonctionnel

Le schéma suivant résume le rôle de chaque bloc dans le montage. Le potentiomètre fournit une consigne de vitesse, le DHT11 fournit les valeurs environnementales, et l'Arduino centralise le traitement avant d'envoyer les sorties vers le LCD et le driver moteur.

Schéma fonctionnel du montage

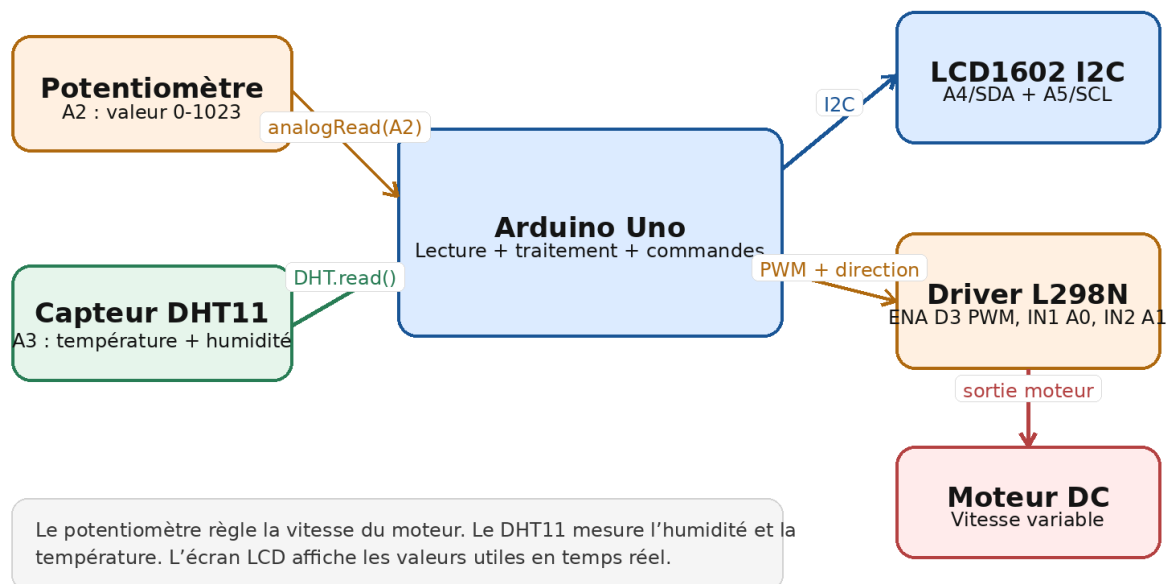


Figure 1 : Schéma fonctionnel du montage.

6. Principe de fonctionnement

Étape	Explication
Initialisation	Le programme démarre la communication série, initialise le capteur DHT11, active l'écran LCD et configure les broches du moteur en sortie.
Lecture de la consigne	La valeur du potentiomètre est lue avec <code>analogRead(POT_PIN)</code> . Cette valeur varie de 0 à 1023.
Conversion en PWM	La fonction <code>map()</code> convertit la valeur 0-1023 en une valeur 0-255, compatible avec <code>analogWrite()</code> .
Commande moteur	La valeur PWM est envoyée sur la broche D3 reliée à ENA du L298N. IN1 est mis à HIGH et IN2 à LOW pour fixer le sens de rotation.
Lecture DHT11	La température et l'humidité sont lues toutes les 2 secondes avec <code>millis()</code> , sans bloquer tout le programme pendant une longue durée.
Affichage	Le LCD affiche la vitesse PWM, la température en degrés Celsius et l'humidité en pourcentage.

7. Explication des parties principales du code

7.1 Bibliothèques utilisées

Instruction / élément	Rôle
Arduino.h	Fonctions de base Arduino : <code>pinMode</code> , <code>digitalWrite</code> , <code>analogRead</code> , <code>analogWrite</code> , <code>delay</code> , <code>millis</code> , etc.
DHT.h	Permet de communiquer avec le capteur DHT11 et de lire l'humidité et la température.
Wire.h	Bibliothèque utilisée pour la communication I2C.
LiquidCrystal_I2C.h	Permet de piloter l'écran LCD1602 à travers le module I2C.

7.2 Initialisation des modules

Instruction / élément	Rôle
<code>LiquidCrystal_I2C lcd(0x27, 16, 2)</code>	Déclare un écran LCD I2C à l'adresse 0x27, avec 16 colonnes et 2 lignes.
<code>DHT dht(DHTPIN, DHTTYPE)</code>	Crée un objet DHT relié à la broche A3 et configuré comme DHT11.
<code>const int pwmA = 3</code>	Broche PWM utilisée pour contrôler la vitesse du moteur via ENA du L298N.

7.3 Fonction setup()

Instruction / élément	Rôle
<code>Serial.begin(9600)</code>	Démarre le moniteur série à 9600 bauds.
<code>dht.begin()</code>	Initialise le capteur DHT11.
<code>lcd.init()</code> et <code>lcd.backlight()</code>	Initialisent l'écran et activent son rétroéclairage.
<code>pinMode(..., OUTPUT)</code>	Configure les broches du moteur comme sorties.
<code>digitalWrite(inA1, HIGH)</code> / <code>digitalWrite(inA2, LOW)</code>	Fixe le sens de rotation du moteur en marche avant.

7.4 Fonction loop()

Instruction / élément	Rôle
<code>analogRead(POT_PIN)</code>	Lit la valeur du potentiomètre entre 0 et 1023.
<code>map(sensorValue, 0, 1023, 0, 255)</code>	Convertit la lecture analogique en valeur PWM.
<code>constrain(motorSpeed, 0, 255)</code>	Garde la valeur de vitesse dans une plage sûre.
<code>analogWrite(pwmA, motorSpeed)</code>	Envoie le signal PWM au driver moteur.
<code>millis()</code>	Permet de lire le DHT11 toutes les 2 secondes sans utiliser un delay long.
<code>lcd.setCursor()</code> et <code>lcd.print()</code>	Positionnent le curseur et affichent les valeurs sur les deux lignes du LCD.

8. Formule de conversion de vitesse

La vitesse envoyée au moteur n'est pas une vitesse réelle en tours par minute. Dans ce montage, elle correspond à la consigne PWM calculée à partir du potentiomètre.

```
motorSpeed = map(sensorValue, 0, 1023, 0, 255)
```

Ainsi, lorsque le potentiomètre est proche de 0, la consigne PWM est proche de 0. Lorsqu'il est proche de 1023, la consigne PWM est proche de 255.

9. Résultats et validation

Test effectué	Résultat attendu / observé	Statut
Démarrage du système	Le LCD affiche « System Ready... » au lancement.	Validé
Variation du potentiomètre	La valeur analogique change et la vitesse PWM envoyée au moteur change.	Validé
Commande du moteur DC	Le moteur tourne dans un sens défini et sa vitesse varie selon la consigne PWM.	Validé

Lecture DHT11	La température et l'humidité sont lues périodiquement.	Validé
Affichage LCD	Le LCD affiche Spd, T et Hum sur deux lignes.	Validé
Moniteur série	Les valeurs Sensor, Speed, Humidité et Température sont affichées pour le diagnostic.	Validé

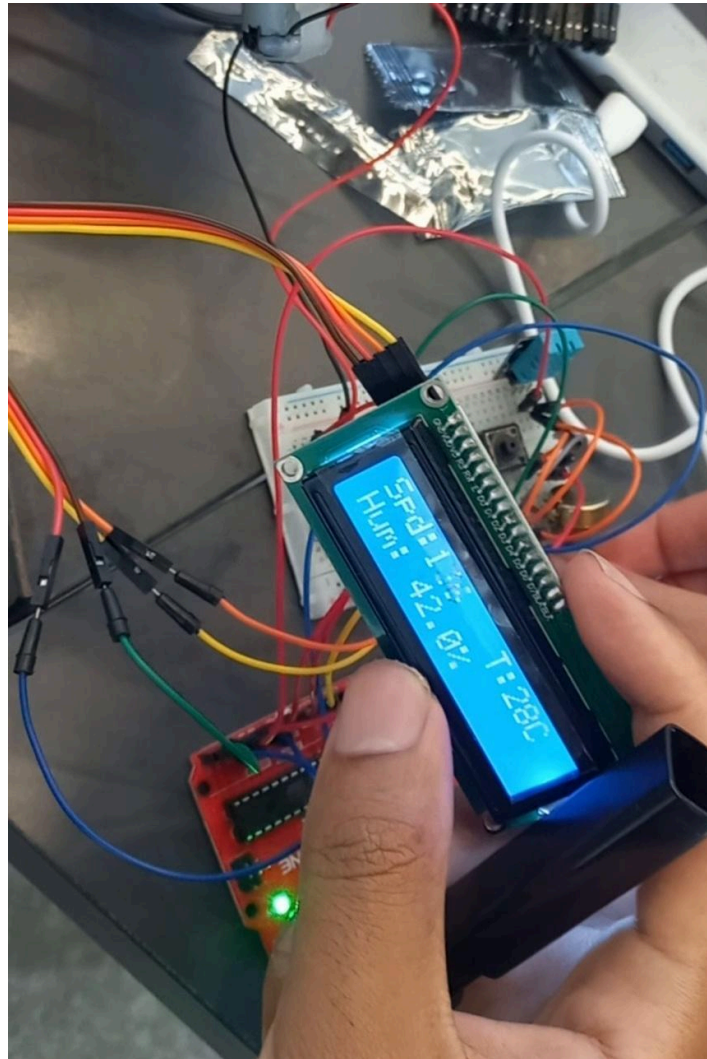


Figure 2 : Capture issue de la vidéo de démonstration du montage.

10. Analyse et remarques techniques

- L'utilisation de l'écran LCD1602 I2C réduit le nombre de fils nécessaires par rapport à un LCD 16 broches classique.
- La broche D3 est choisie pour la vitesse car elle supporte la sortie PWM sur Arduino Uno.
- Le programme utilise millis() pour éviter une attente longue entre deux lectures du DHT11.
- La valeur affichée sous le nom Spd correspond à la valeur PWM, pas à une mesure de vitesse réelle du moteur.
- L'ajout d'un encodeur pourrait permettre de mesurer la vitesse réelle du moteur en RPM.
- Une alimentation externe adaptée au moteur est recommandée pour éviter une surcharge de l'Arduino.

11. Difficultés possibles et solutions

Problème possible	Solution proposée
LCD vide ou illisible	Vérifier l'adresse I2C 0x27, les fils SDA/SCL et le contraste du module I2C.
Moteur ne tourne pas	Vérifier l'alimentation du moteur, ENA, IN1/IN2 et la masse commune.
Valeurs DHT non lues	Vérifier la broche DATA, le type DHT11 et attendre entre deux lectures.
Affichage avec anciens caractères	Ajouter des espaces après les valeurs affichées pour effacer les anciens chiffres.
Valeur PWM incorrecte	Vérifier le potentiomètre, la broche A2 et le câblage 5V/GND/signal.

12. Conclusion

Ce montage permet de combiner la lecture d'une entrée analogique, la commande d'un moteur DC, la mesure de température et d'humidité, et l'affichage local des données. Le système fonctionne comme une petite application embarquée complète : acquisition, traitement, commande et visualisation.

La réalisation montre l'importance de bien organiser les broches, de séparer les lectures capteurs des actions de commande, et d'utiliser le moniteur série ainsi que l'écran LCD pour vérifier le comportement du système pendant les tests.

13. Références consultées

- Arduino Docs - analogWrite() : <https://docs.arduino.cc/language-reference/en/functions/analog-io/analogWrite/>
- Arduino - Liquid Crystal Displays with Arduino : <https://www.arduino.cc/en/Tutorial/HelloWorld>
- Arduino Docs - Inter-Integrated Circuit / Wire Library : <https://docs.arduino.cc/learn/communication/wire>
- Arduino Libraries - DHT sensor library : <https://docs.arduino.cc/libraries/dht-sensor-library/>
- STMicroelectronics - L298 Dual Full-Bridge Driver Datasheet : <https://www.st.com/resource/en/datasheet/l298.pdf>

Annexe A - Code source complet

```
#include <Arduino.h>
#include "DHT.h"
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

LiquidCrystal_I2C lcd(0x27, 16, 2);

// DHT Sensor
#define DHTPIN A3      // Broche à laquelle le capteur est connecté
#define DHTTYPE DHT11  // Définir le type de capteur
DHT dht(DHTPIN, DHTTYPE);

// Motor A pins
const int pwmA = 3;    // ENA pin on L298N, must be PWM pin
const int inA1 = A0;   // IN1
const int inA2 = A1;   // IN2

// Potentiometer pin
const int POT_PIN = A2; // Middle pin of potentiometer

void setup() {
  Serial.begin(9600);
  dht.begin();
```

```

lcd.init();          // Start LCD
lcd.backlight();     // Turn on backlight
lcd.setCursor(0, 0);
lcd.print("System Ready...");

pinMode(pwmA, OUTPUT);
pinMode(inA1, OUTPUT);
pinMode(inA2, OUTPUT);

// Direction forward
digitalWrite(inA1, HIGH);
digitalWrite(inA2, LOW);
}

void loop() {
  // Read potentiometer value: 0 to 1023
  int sensorValue = analogRead(POT_PIN);

  // Convert potentiometer value to motor speed: 0 to 255
  int motorSpeed = map(sensorValue, 0, 1023, 0, 255);

  // Safety limit
  motorSpeed = constrain(motorSpeed, 0, 255);

  // Send PWM speed to motor
  analogWrite(pwmA, motorSpeed);

  // Serial monitor
  Serial.print("Sensor: ");
  Serial.print(sensorValue);
  Serial.print(" | Speed: ");
  Serial.println(motorSpeed);

  // Read DHT sensor non-blocking (every 2 seconds)
  static unsigned long lastDHTRead = 0;
  static float lastH = 0.0;
  static float lastT = 0.0;

  if (millis() - lastDHTRead >= 2000) {
    lastDHTRead = millis();
    float h = dht.readHumidity();
    float t = dht.readTemperature();

    if (isnan(h) || isnan(t)) {
      Serial.println("Échec de lecture !");
    } else {
      lastH = h;
      lastT = t;
      Serial.print("Humidité: ");
      Serial.print(h);
      Serial.print(" %\t");
      Serial.print("Température: ");
      Serial.print(t);
      Serial.println(" *C");
    }
  }

  // Update LCD Display
  lcd.setCursor(0, 0);
  lcd.print("Spd:");
  lcd.print(motorSpeed);
  lcd.print("   T:");
  lcd.print((int)lastT);
  lcd.print("C   "); // Padding to overwrite remaining digits

  lcd.setCursor(0, 1);
  lcd.print("Hum: ");
  lcd.print(lastH, 1);
  lcd.print("%    ");

  delay(100);
}

```